VERIQTA

COMPLETE 20-PAGE HANDBOOK

★ PRACTICAL. VISUAL. INTERVIEW READY. ★

KUBERNETES HANDBOOK

THE ULTIMATE QUICK REFERENCE GUIDE

✓ FOR BEGINNERS | ✓ CKA / CKAD / INTERVIEWS | ✓ DEVOPS & CLOUD ENGINEERS

FROM CONCEPTS TO PRODUCTION!

INTERVIEW REAL-WORLD EXAMPLES FOCUSED

- CORE CONCEPTS
- WORKLOADS & CONTROLLERS
- NETWORKING & SERVICES
- SECURITY & POLICIES
- SCALING & PERFORMANCE
- OPERATIONS & TROUBLESHOOTING
- CI/CD & DEVOPS
- INTERVIEW QUESTIONS

✓ YAML EXAMPLES | KUBECTL COMMANDS | INTERVIEW QUESTIONS | BEST PRACTICES

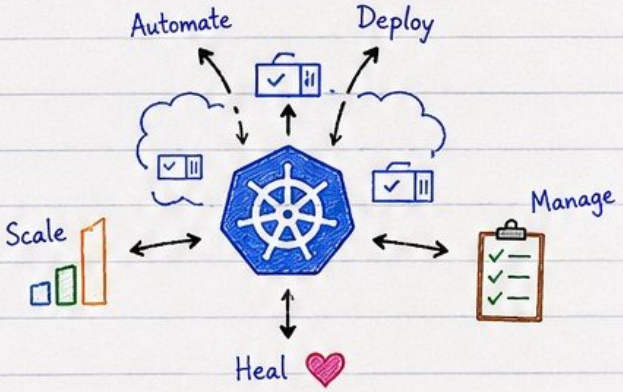
MASTER KUBERNETES. BUILD THE FUTURE.

Introduction To Kubernetes

1 What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications.

☆ K8s helps you run your applications at scale, with high availability.



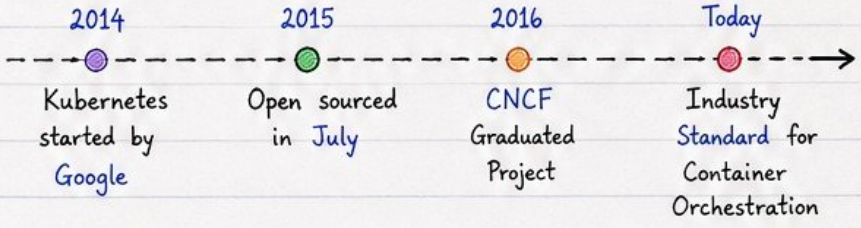
2 Why Kubernetes was created?

- To manage containerized applications at scale.
- To automate deployment, scaling, and operations.
- To ensure high availability and fault tolerance.
- To provide a consistent platform across different environments (on-prem, cloud, hybrid).
- To optimize resource utilization.

Problem Before K8s

- × Manual deployments
- × Hard to scale
- × Poor resource utilization
- × Downtime on failures
- × Environment inconsistency

3 History of Kubernetes



☆ K8s has the largest ecosystem and is backed by CNCF.

4 Google Borg Inspiration

- Kubernetes is inspired by Google's internal system "Borg".
- Borg was a cluster manager for running large-scale distributed workloads.
- K8s brings Borg-like power to everyone!

Borg (Google)
- Internal
- Proprietary
- For Google apps
- Massive scale

→

Kubernetes
- Open Source
- For Everyone
- Runs anywhere
- Extensible

5 Benefits of Container Orchestration with Kubernetes

Automation	Scalability	High Availability	Efficiency	Portability	Extensibility
Less manual work	Scale up/down easily	Self-healing & failover	Better resource utilization	Run anywhere (cloud/on-prem)	Large ecosystem & integrations

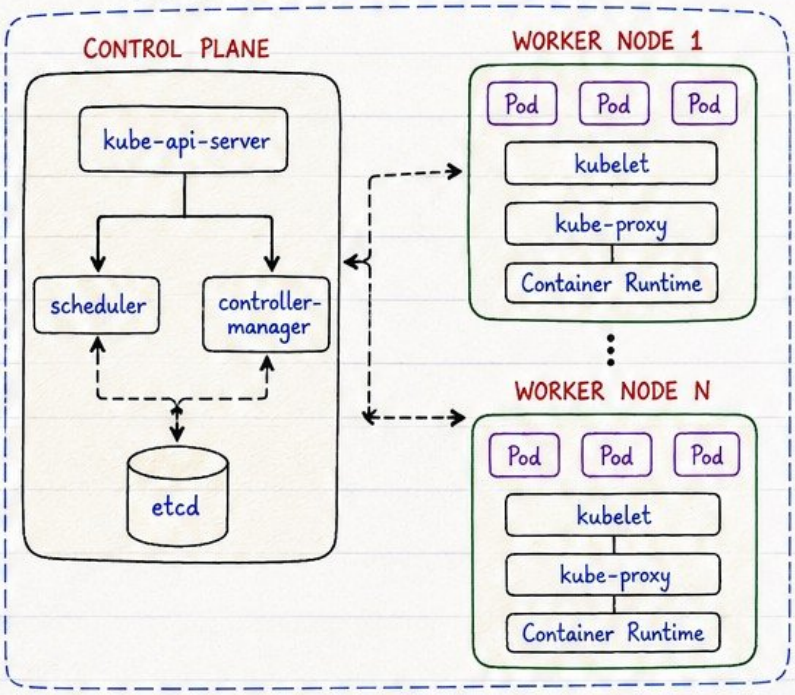
☆ Kubernetes makes it easy to run modern, cloud-native applications reliably and efficiently! ♡

Kubernetes Architecture Overview

1 High Level Overview

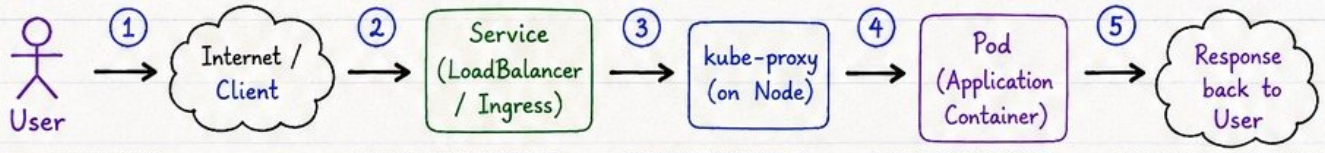
- Kubernetes follows a Master-Worker architecture.
- The Cluster is managed by the Control Plane (Master).
- Applications run on Worker Nodes as containers inside Pods.
- The Control Plane makes global decisions and detects & responds to cluster events.
- Worker Nodes run your applications and report status back to the Control Plane.

Kubernetes Cluster Architecture



→ Request / Response
--→ Watch / Events
--→ Traffic Flow

2 Request Flow



User sends request to the application (e.g. via browser or API client)

Request reaches the Service (via Ingress or LoadBalancer) which exposes the application.

kube-proxy forwards the request to a healthy Pod (via iptables or IPVS).

The Pod processes the request and sends the response back.

Response travels back through the same path to the user.

3 Control Plane (Master)

- ★ Manages the cluster.
- ★ Makes global scheduling decisions.
- ★ Detects and responds to cluster events.
- ★ Maintains the desired state of the cluster.
- ★ Does not run user applications.

Think of Control Plane as the BRAIN 🧠 of the cluster.

4 Worker Nodes

- ★ Run your application workloads.
- ★ Hosts Pods (one or more).
- ★ Communicate with Control Plane.
- ★ Monitor Pod health via kubelet.
- ★ Provide networking and storage.

Think of Worker Nodes as the HANDS & LEGS 💪 of the cluster.

★ Kubernetes is designed to be highly available, extensible, resilient and declarative. You describe the desired state, K8s makes sure it is always achieved! ♡

Control Plane Components

The Control Plane (Master) makes global decisions about the cluster and detects & responds to cluster events.

1 API Server (kube-apiserver)

- Frontend for the Kubernetes Control Plane.
- Exposes the Kubernetes API.
- Validates and processes REST requests.
- All components communicate through the API Server.

2 Scheduler (kube-scheduler)

- Watches for newly created Pods with no assigned node.
- Selects the best node for the Pod based on resources, policies, constraints.
- Binds the Pod to the chosen node via the API Server.

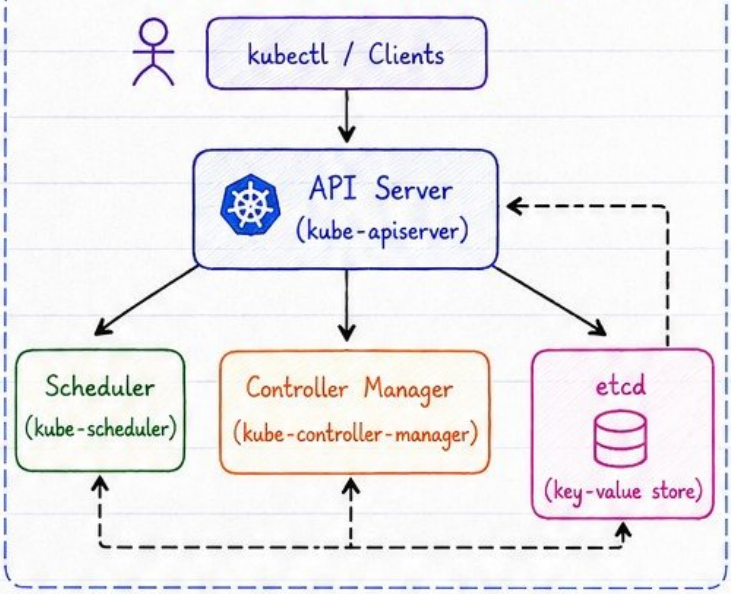
3 Controller Manager (kube-controller-manager)

- Runs system controllers.
- Ensures the actual state matches the desired state.
- Examples: Node Controller, ReplicaSet Controller, Endpoints Controller, Service Controller, etc.

4 ETCD

- Consistent and highly-available key-value store.
- Stores all cluster data and configuration.
- Single source of truth for the entire cluster.
- Backup is critical!

CONTROL PLANE ARCHITECTURE



HOW THEY WORK TOGETHER

- ☑ API Server is the entry point for all requests.
- ☑ Scheduler decides where new Pods should run.
- ☑ Controller Manager continuously monitors and moves the cluster to the desired state.
- ☑ etcd stores every piece of data about the cluster.

COMPONENT SUMMARY

Component	Main Role	Key Responsibilities	Communicates With
API Server (kube-apiserver)	Exposes Kubernetes API & handles all requests.	• Authentication & Authorization • Validates requests • Updates and reads from etcd	All components, etcd, kubectl, clients, nodes
Scheduler (kube-scheduler)	Decides where Pods should run.	• Watches for unscheduled Pods • Evaluates nodes • Binds Pod to the best node	API Server, etcd, kubelet
Controller Manager (kube-controller-manager)	Runs controllers that maintain cluster state.	• Node Controller • ReplicaSet Controller • Service Controller, etc.	API Server, etcd, all controllers
ETCD (key-value store)	Stores all cluster data persistently.	• Stores configuration & state • Provides strong consistency • Critical for cluster operation	API Server, all control plane components

If etcd goes down, the cluster cannot read or write data. Always backup etcd and monitor its health.

Worker Node Components

Worker Nodes are the machines that run your application workloads (Pods). They communicate with the Control Plane and keep Pods running.

1 Kubelet

- Agent that runs on each node.
- Ensures containers are running in a Pod.
- Communicates with API Server.
- Performs liveness & readiness checks.

2 Kube Proxy

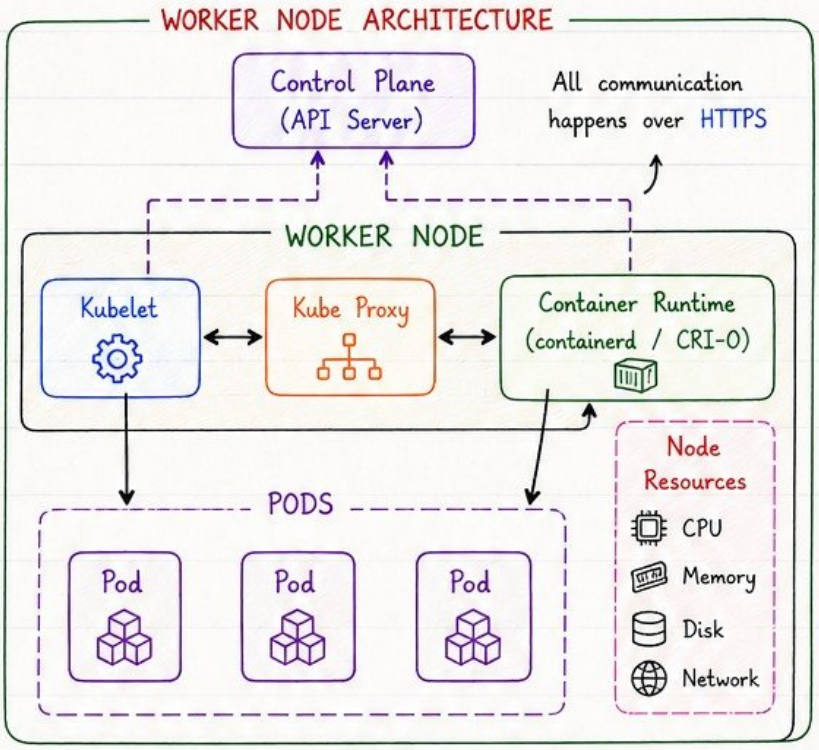
- Network proxy that runs on each node.
- Maintains network rules.
- Enables communication to Pods from inside & outside the cluster.

3 Container Runtime

- Software responsible for running containers.
- Examples: containerd, CRI-O, Docker (deprecated).
- Pulls images & starts/stops containers.

4 Pods (on Node)

- Ssmallest deployable units.
- Contains one or more containers that share network & storage.
- Scheduled by the Control Plane to the node.



COMMON CONTAINER RUNTIMES

containerd

- CNCF graduated
- Lightweight
- Default in most distros

CRI-O

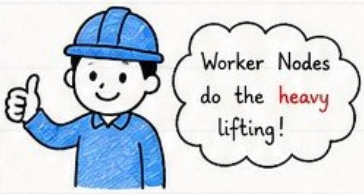
- Lightweight
- Designed for K8s
- No Docker daemon

Docker (deprecated)

- Heavyweight
- Dockershim removed in K8s 1.24+

HOW IT WORKS

1. Scheduler assigns a Pod to a Node.
2. Kubelet on the node receives the Pod spec.
3. Kubelet pulls the image via Container Runtime.
4. Container Runtime starts the containers.
5. Kubelet reports status back to API Server.



WORKER NODE COMPONENTS SUMMARY

Component	Main Role	Communicates With
Kubelet	Ensures Pods are running as expected.	API Server, Container Runtime
Kube Proxy	Handles networking and service routing.	API Server, Services, Pods
Container Runtime	Runs containers (pull, start, stop).	Kubelet, Container Registry
Pods	Run your application containers.	Kubelet, Services, Other Pods

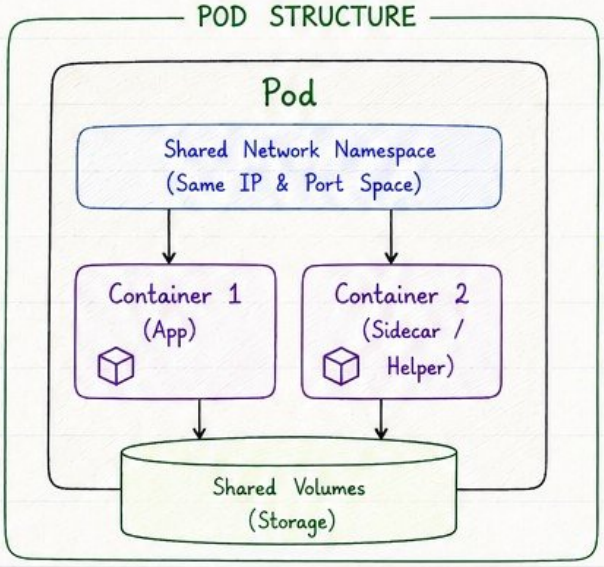
A healthy Worker Node + happy components = happy workloads!

Pods Explained

A Pod is the smallest deployable unit in Kubernetes. It represents a single instance of a running process in your cluster.

1 What is a Pod?

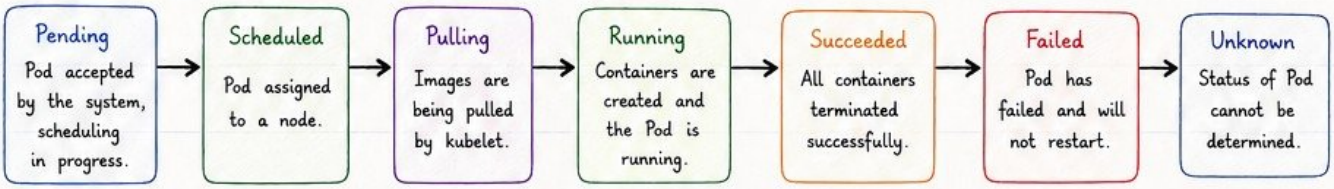
- A Pod encapsulates one or more containers, storage resources, a unique network IP, and options that govern how the container(s) should run.
- Containers in a Pod share the same network namespace and storage volumes.
- Pods are ephemeral – they can be created, destroyed, and replaced.



Examples:

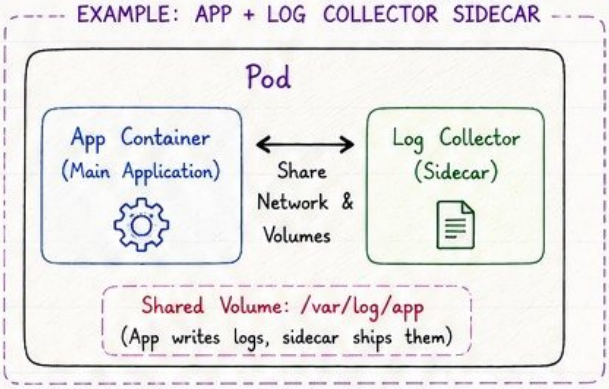
- Main App + Log Collector
- App + Config Loader
- App + Proxy (Sidecar)

2 Pod Lifecycle



3 Multi-container Pods

- Pods can run multiple containers that work together.
- Common pattern: Sidecar container for logging, monitoring, or proxy.
- All containers in a Pod share:
  - ☑ Same network namespace (IP & ports)
  - ☑ Same volumes



Why Sidecar?

- Extends or enhances the main container.
- Keeps containers loosely coupled.
- Reusable & independent.

4 Pod States

State	Description	Details
Pending	Pod is accepted but not yet scheduled.	Waiting for a node to be assigned.
Running	Pod is bound to a node and all containers are running.	Pod is in healthy state.
Succeeded	All containers in the Pod terminated successfully.	Used for Jobs / batch workloads.
Failed	At least one container in the Pod terminated in failure.	Pod will not be restarted.
Unknown	Kubelet cannot report the status of the Pod.	Usually due to node or network issues.

5 Key Takeaways

- Pod is the smallest unit in K8s.
- Pods are ephemeral and replaceable.
- Each Pod gets its own IP address.
- Containers in a Pod share network and storage.
- Design Pods to run a single application (or tightly-coupled containers).

Think of a Pod as a "logical host" for your containers – they live, work, share resources, and die together.

REPLICASETS

ReplicaSet ensures that a specified number of Pod replicas are running at any given time. It is the foundation for Deployments.

1 Purpose

- Ensures a stable set of replica Pods are running at all times.
- Maintains the desired number of replicas.
- Replaces failed or deleted Pods.
- Does not handle updates (use Deployments for that).

2 How ReplicaSets Work

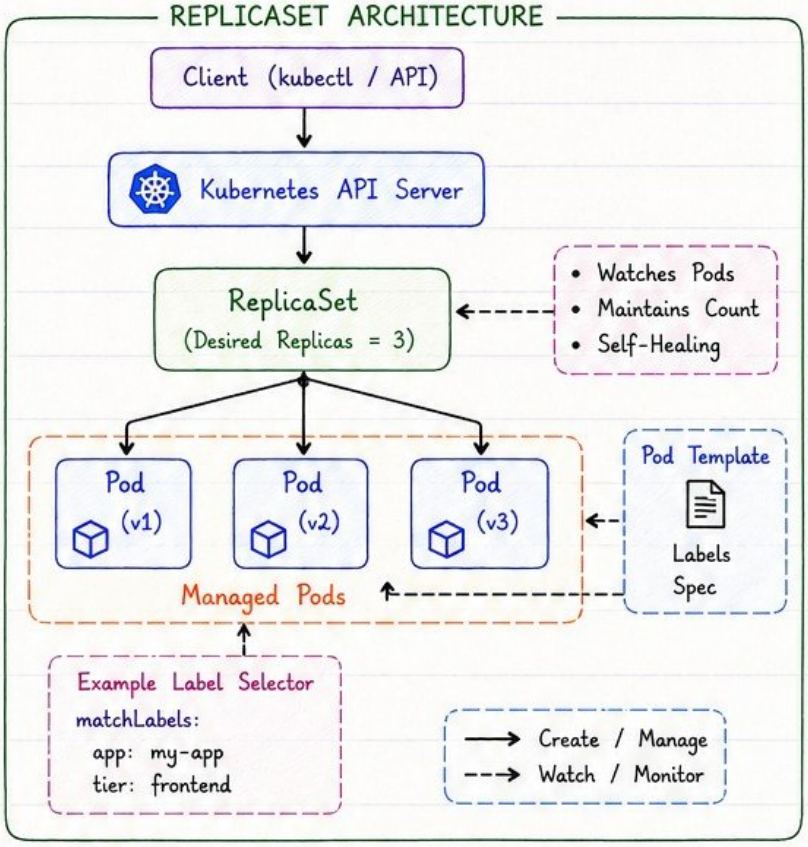
- You define a Pod template and the desired number of replicas.
- ReplicaSet creates Pods to match that desired state.
- It continuously monitors the Pods.
- If a Pod fails or is deleted, it creates a new one to maintain the count.

3 Scaling

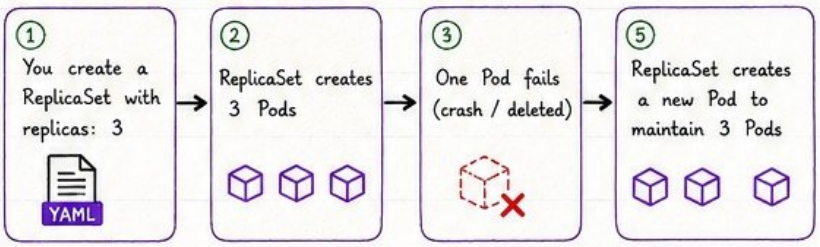
- You can scale up or down by updating the replica count.
- Scaling can be done manually or via Horizontal Pod Autoscaler (HPA).

4 Self-Healing

- If a Pod crashes, gets deleted, or the node fails, ReplicaSet ensures the desired number of Pods are always running.
- Provides high availability and reliability.



5 ReplicaSet in Action (Example)



6 YAML Example

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
  labels:
    app: my-app
spec:
  replicas: 3       # desired number of replicas
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80
```

WHAT IT MEANS

- replicas: 3 → Keep 3 Pods running at all times.
- selector → Tells ReplicaSet which Pods to manage.
- template → Defines how the Pods should look.

7 Key Points to Remember

- ReplicaSet does not handle rolling updates (use Deployments).
- Use meaningful labels for selectors.
- ReplicaSet is immutable for selector – don't change it.
- It is the backbone of Deployments and advanced workloads.
- Keeps your applications constant, available & resilient.

ReplicaSet = Desired State + Continuous Monitoring + Self-Healing = High Availability

DEPLOYMENTS

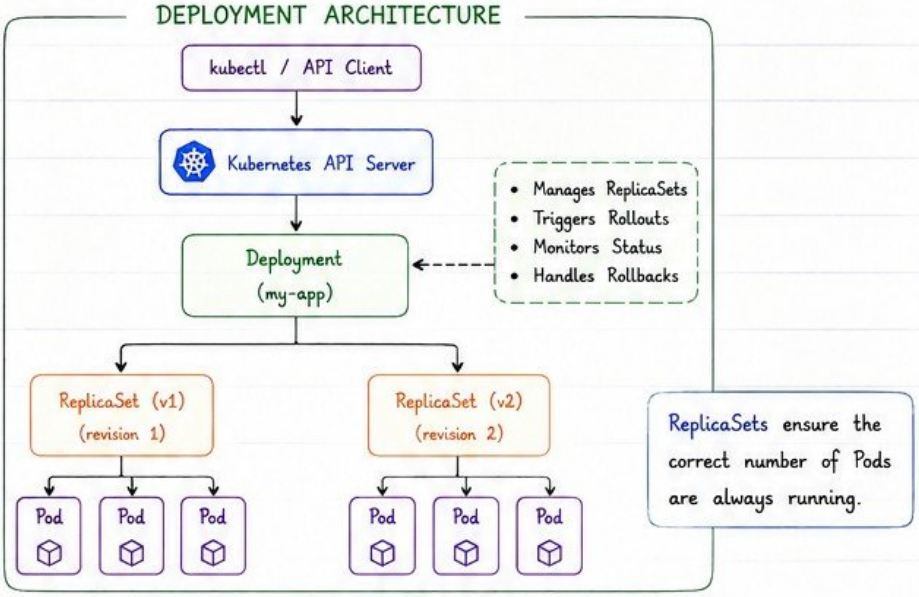
A Deployment provides declarative updates for Pods and ReplicaSets. It allows you to roll out updates, roll back, and manage the desired state.

1 What is a Deployment?

- Manages ReplicaSets and Pods.
- Provides declarative updates for your applications.
- Ensures the desired number of Pods are running.
- Supports rollouts, rollbacks and pause/resume.
- Ideal for stateless applications.

Think of Deployment as the "Manager" and ReplicaSet as the "Supervisor".

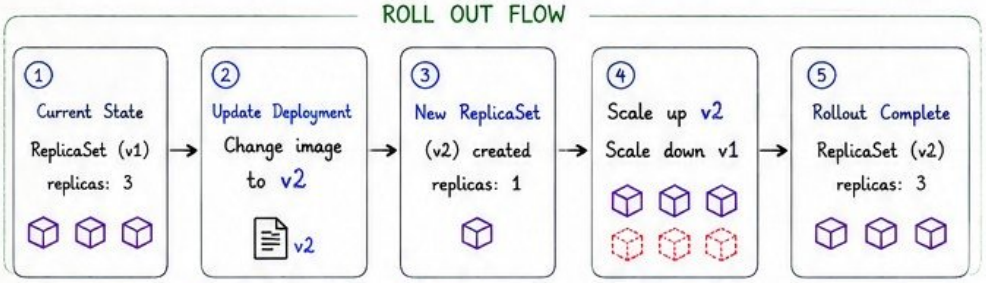
DEPLOYMENT ARCHITECTURE



2 Rollout (Update)

- Update the image or configuration in the Deployment.
- Kubernetes creates a new ReplicaSet with the new version.
- Gradually shifts traffic to new Pods.
- Old ReplicaSet is scaled down.

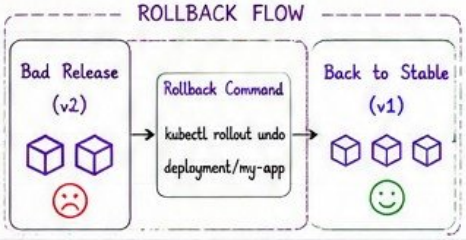
ROLL OUT FLOW



3 Rollback

- If something goes wrong, roll back to a previous ReplicaSet.
- Kubernetes instantly scales up the old ReplicaSet and scales down the bad one.
- Rollback is fast and reliable.

ROLLBACK FLOW



4 Update Strategies

- Recreate: Terminates old Pods before creating new ones (downtime).
- RollingUpdate (default): Gradually replaces Pods with zero/minimal downtime.
- Blue/Green & Canary: Advanced strategies for safer releases.

5 Deployment Commands

```
# Create Deployment
kubectl create deployment my-app --image=nginx
# Check status
kubectl rollout status deployment/my-app
# Update image
kubectl set image deployment/my-app nginx=nginx:1.25
# Rollback
kubectl rollout undo deployment/my-app
# Rollback to specific revision
kubectl rollout undo deployment/my-app --to-revision=2
# History
kubectl rollout history deployment/my-app
kubectl rollout history deployment/my-app --revision=2
# Pause / Resume
kubectl rollout pause deployment/my-app
kubectl rollout resume deployment/my-app
```

6 Key Features Summary

Feature	Description	Benefit
Declarative Updates	You define the desired state, K8s makes it happen.	Less manual work, fewer errors.
Rollouts	Update with zero or minimal downtime.	Better user experience.
Rollbacks	Quickly revert to a known good state.	High reliability & faster recovery.
Scaling	Scale up/down easily via replica count or HPA.	Handle load efficiently.
Self-Healing	Automatically replaces failed Pods.	Always-on applications.

Deployments make your applications resilient, updatable, and easy to manage. Use Deployments for stateless apps and let Kubernetes handle the rest!

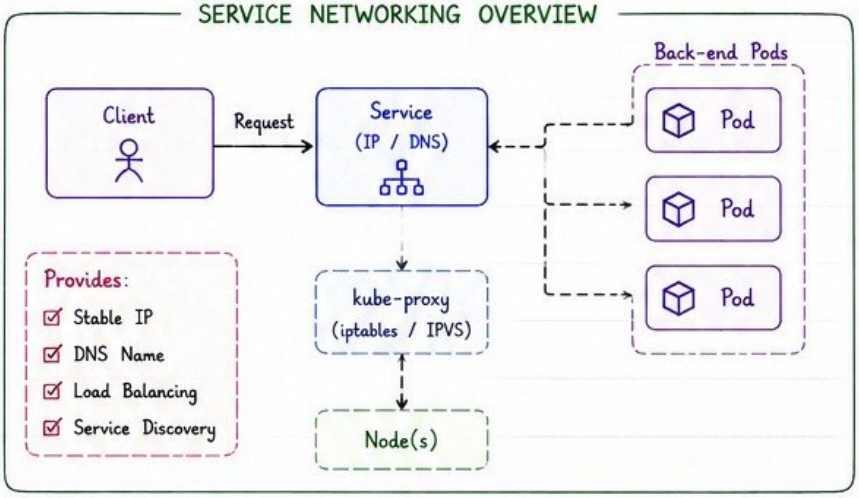
SERVICES

☆ A Service is an abstraction which defines a logical set of Pods and a policy by which to access them (stable IP & DNS).

1 What is a Service?

- Pods are ephemeral (IP can change).
- Services provide a stable IP and DNS name to access Pods.
- Acts as a load balancer across Pods.
- Decouples clients from Pods.

Think: Service = Stable endpoint for a group of Pods (today or tomorrow)



2 Types of Services

Type	How it Works	Use Case	Access	Diagram
ClusterIP (Default)	Exposes the Service on a cluster-internal IP.	Internal communication within the cluster.	Only from within the cluster.	Pod → ClusterIP → Pod
NodePort	Exposes the Service on each Node's IP at a static port (30000-32767).	Access from outside using <NodeIP>:<NodePort>.	From outside via Node IP and NodePort.	External Client → NodeIP:NodePort (30000-32767) → Service
LoadBalancer	Creates an external cloud provider load balancer that routes to the Service.	Expose to Internet using cloud load balancer.	From outside via Cloud LB IP.	External Client → Cloud Load Balancer → Service
ExternalName	Maps the Service to the contents of an external DNS name (CNAME).	Access external services outside the cluster.	Resolves to external DNS name.	Pod → ExternalName (CNAME) → external.example.com (External Service)

3 Service Selection

- Services use labels and selectors to find the right Pods.
- Only Pods matching the selector become endpoints of the Service.

Example Service YAML (ClusterIP)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
    - port: 80          # Service Port
      targetPort: 8080  # Pod Port
```

Selector matches Pods with label app: my-app

4 How It Works (Flow)

1. Client sends request to Service IP or DNS
2. kube-proxy intercepts the request
3. Request is forwarded to one of the healthy Pods
4. Pod processes the request and sends response
5. Response goes back to the client via kube-proxy

☆ kube-proxy ensures load balancing and network rules (iptables / IPVS).

5 Key Takeaways

☆ Services give stability to dynamic Pods.
☆ Use ClusterIP for internal apps.
☆ Use NodePort for simple external access.
☆ Use LoadBalancer for production (cloud).
☆ Use ExternalName for external DNS mapping.
☆ Labels & Selectors are the key to Service discovery.

Services are the bridge between the world and your Pods!

Services hide complexity and expose simplicity.
Right Service type + Right selector = Happy Users!

NAMESPACES

Multi-Tenant

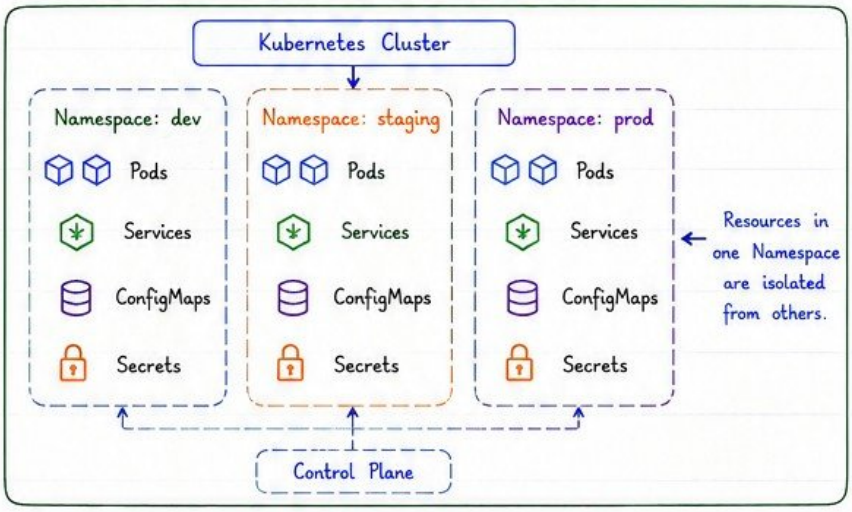
A Namespace provides a way to divide cluster resources between multiple users, teams, or projects. It helps in resource isolation.

1 What is a Namespace?

- Namespaces partition the cluster resources.
- Resources in one Namespace are isolated from others.
- Each resource (Pod, Service, etc.) belongs to a Namespace.
- Default Namespaces: default, kube-system, kube-public.

Think: Namespaces are like folders in a file system – they help organize and isolate.

2 Resource Isolation with Namespaces



3 Why Use Namespaces?

- Resource Isolation
- Multi-tenancy
- Better Organization
- Quota & Limit Management
- Access Control (RBAC per Namespace)
- Avoid Naming Conflicts

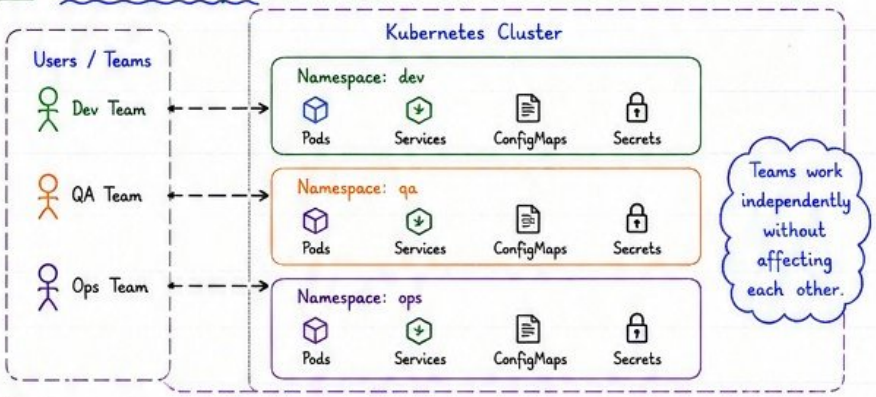
4 Common Use Cases

Development Teams	Environment Separation	Projects Isolation	Third-party Access	System Components
Each team gets their own Namespace.	dev, staging, test, prod in separate Namespaces.	Different projects isolated from each other.	Give access to vendors in a separate Namespace.	System add-ons run in kube-system Namespace.

5 Default Namespaces

Namespace	Purpose
default	Default Namespace for resources without a specific Namespace.
kube-system	Used by Kubernetes system components.
kube-public	Publicly readable (mostly for cluster info).
kube-node-lease	Used for node heartbeats.

6 Visual Example



7 Best Practices

- ☑ Create a Namespace per team or project.
- ☑ Apply Resource Quotas to prevent overuse.
- ☑ Use RBAC to control access per Namespace.
- ☑ Avoid using the default Namespace for production.
- ☑ Keep cluster-wide components in kube-system.

8 Useful Commands

```
# Create Namespace
kubectl create namespace dev                                  # creates a Namespace
# List Namespaces
kubectl get namespaces                                        # lists all Namespaces
# Get resources in a Namespace
kubectl get pods -n dev                                       # shows Pods in 'dev' Namespace
# Set current Namespace
kubectl config set-context --current --namespace=dev          # switch context to a Namespace
# Delete Namespace
kubectl delete namespace dev                                  # deletes the Namespace
```

Namespaces help you stay organized, secure, and scalable in Kubernetes.
Right isolation today, fewer headaches tomorrow!

CONFIGMAPS AND SECRETS

ConfigMaps and Secrets help you manage configuration and sensitive data separately from your container images.

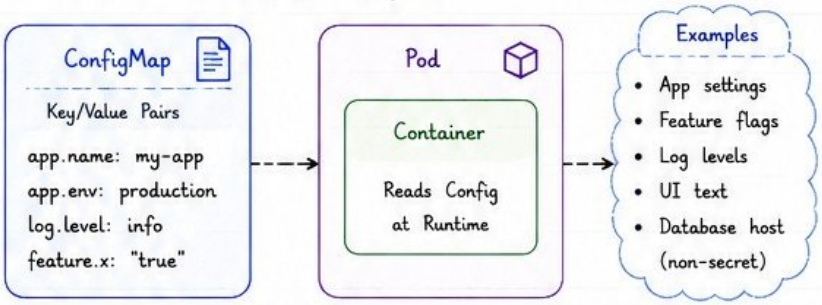
1 Configuration Management

- Keep configuration data outside the container image.
- Make applications portable and easy to manage.
- Update configuration without rebuilding images.
- ConfigMaps for non-sensitive data.
- Secrets for sensitive data.

Think: Don't bake config into images, manage it separately!

2 What Are ConfigMaps?

- Store non-sensitive key-value pairs, files, or properties.
- Used for environment-specific configuration.

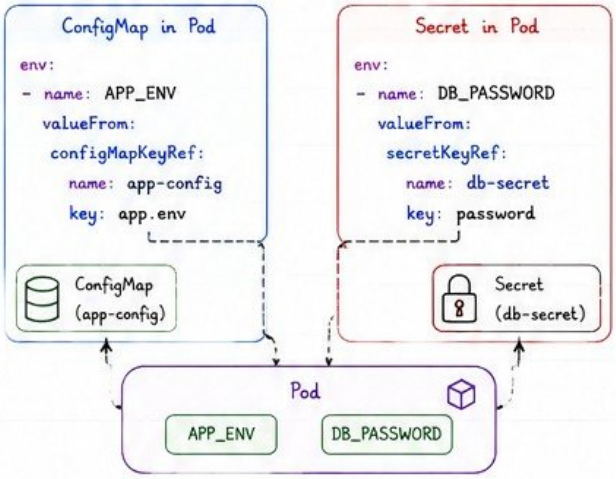


ConfigMaps are not encrypted. Do not store sensitive data!

3 What Are Secrets?

- Store sensitive data such as passwords, tokens, keys, and certificates.
- Stored in Base64 encoded format (by default).
- Can be encrypted at rest (etcd encryption) for better security.

4 How They Are Used in Pods



5 Creating ConfigMaps and Secrets

ConfigMap (from literal)

```
kubectl create configmap app-config \
  --from-literal=app.name=my-app \
  --from-literal=app.env=production
```

ConfigMap (from file)

```
kubectl create configmap app-config \
  --from-file=app.properties
```

Secret (from literal)

```
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=Passw0rd!
```

Secret (from file)

```
kubectl create secret generic tls-secret \
  --from-file=tls.crt=cert.crt \
  --from-file=tls.key=key.key
```

6 Key Differences

Feature	ConfigMap	Secret
Purpose	Store non-sensitive configuration	Store sensitive data
Data Type	Key/Value, Files	Key/Value, Files
Encoding	Plain text	Base64 encoded
Security	Not encrypted	Can be encrypted (at rest)
Use Cases	App settings, feature flags, env config	Passwords, tokens, certificates, keys

7 Best Practices

- ☑ Use ConfigMaps for all non-sensitive configuration.
- ☑ Use Secrets for all sensitive data.
- ☑ Enable etcd encryption for Secrets.
- ☑ Use RBAC to restrict access to Secrets.
- ☑ Avoid storing Secrets in environment variables if possible.
- ☑ Use external secret stores for high-security environments.

Pro Tip

Combine ConfigMaps and Secrets to keep your apps flexible and secure.

8 Useful Commands

```
# List ConfigMaps
kubectl get configmaps
# Describe ConfigMap
kubectl describe configmap app-config
# List Secrets
kubectl get secrets
# Describe Secret
kubectl describe secret db-secret
```

Externalize your configuration. Protect your secrets.
Secure today, scale tomorrow! ☺

Labels and Selectors

Labels and Selectors are the core of how Kubernetes organizes and finds resources. Use them consistently and meaningfully.

1 What Are Labels?

- Labels are key/value pairs attached to Kubernetes objects.
- They do not have to be unique.
- Used to organize and identify subsets of objects.
- Can be added to Pods, Services, Deployments, Namespaces, etc.

Labels describe what an object is.
Selectors find what matches.

2 Label Anatomy

key = value

- Prefix (optional) — e.g., app.kubernetes.io/name
- Name (required) — e.g., my-app

Label Rules

- Max 63 characters
- Lowercase letters, numbers, '-', '_', or '.'
- Must start and end with an alphanumeric character

3 Common Labels

Label	Purpose
app	Identifies the application
env	Identifies the environment (dev, staging, prod)
tier	Identifies the tier (frontend, backend, db)
version	Identifies the version
team	Owner team
component	Part of a larger system

Examples

```
app: my-app
env: production
tier: frontend
version: v2
team: payments
component: api
```

4 What Is a Selector?

- Selectors are used to find a set of objects based on their labels.
- Used by Services, Deployments, NetworkPolicies, ReplicaSets, and more.
- Two types: Equality-based and Set-based.

Selector Types

Equality-based
Match a label with exact key/value.
Example: `app: my-app`

Set-based
More advanced matching (in, notin, exists, notexists).
Example: `env in (dev, staging)`

5 Selector Operators (Set-based)

Operator	Meaning	Example
=	Equal	env = production
!=	Not equal	tier != backend
in	In a set	env in (dev, staging, prod)
notin	Not in a set	env notin (dev, test)
exists	Key exists	version
!exists	Key does not exist	!debug

Set-based selectors are more powerful and flexible.

6 Selector Examples

Equality-based Selector

```
selector:
  matchLabels:
    app: my-app
    env: production
```

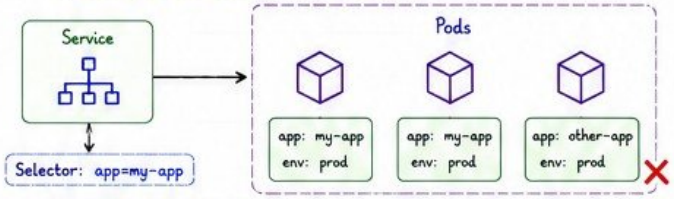
Matches objects with: app=my-app AND env=production

Set-based Selector

```
selector:
  matchExpressions:
  - key: env
    operator: in
    values: [dev, staging]
```

Matches objects with: env=dev OR env=staging

7 How Services Use Selectors



Service sends traffic only to Pods that match the selector (app=my-app).

8 Best Practices

- Use meaningful and consistent label keys.
- Use recommended labels (app.kubernetes.io/*).
- Keep label values simple and lowercase.
- Do not use labels that change frequently.
- Keep label cardinality low (avoid high uniqueness).
- Always test your selectors!

9 YAML Example

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app-pod
  labels:
    app: my-app
    env: production
    tier: frontend
```

Remember:
Labels are on the object.
Selectors are used to find them.

10 Key Takeaways

- ☑ Labels identify and organize Kubernetes objects.
- ☑ Selectors find objects based on labels.
- ☑ Use equality for simple matching, set-based for advanced.
- ☑ Good labels = easy management and reliable applications.
- ☑ Plan your labeling strategy early!

Great labels and selectors = Organized clusters, happy users, and scalable applications!
Label today. Automate tomorrow. Scale forever. ☺

Ingress – Exposing Services to the World

Ingress

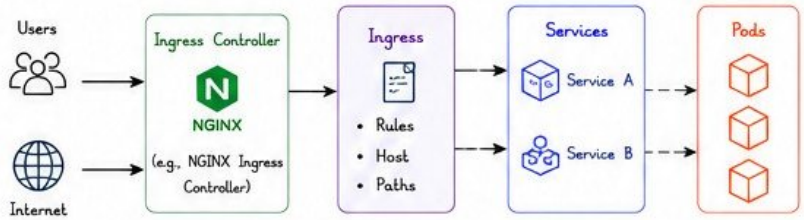
> Ingress provides HTTP/HTTPS routing to Services based on hostnames and paths. It is managed by an Ingress Controller.

1 What is Ingress?

- Ingress exposes HTTP and HTTPS routes from outside the cluster to Services.
- Provides load balancing, SSL termination, name-based virtual hosting, and path-based routing.
- Works with an Ingress Controller (e.g., NGINX Ingress Controller).

Think:
Ingress = Traffic entry point for your applications.

2 Ingress Architecture



> Ingress Controller watches Ingress resources and configures the load balancer (or reverse proxy) to route traffic to the right Service.

3 Ingress Resource

- An Ingress object defines rules to route HTTP/HTTPS traffic.
- Rules are based on hostnames and URL paths.

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app.veriqta.com
    http:
      paths:
      - path: /app1
        pathType: Prefix
        backend:
          service:
            name: app1-service
            port:
              number: 80
      - path: /app2
        pathType: Prefix
        backend:
          service:
            name: app2-service
            port:
              number: 80
```

Explanation

- **host:** The domain name.
- **paths:** URL paths to match.
- **pathType:**
  - Prefix (default)
  - Exact
  - ImplementationSpecific
- **backend:** The Service to send traffic to.

Tip
Host + Path = Routing Rule
Ingress does not create a Service. It routes traffic to existing Services.

4 Ingress Controllers

- Ingress Controllers implement the Ingress rules.
- Popular Ingress Controllers:

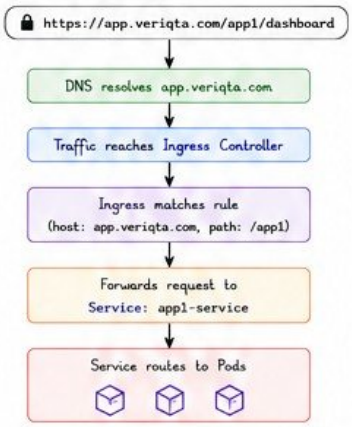
Ingress Controller	Description	Features
NGINX Ingress Controller	Most popular open-source controller.	High performance, rich annotations, SSL, WAF support.
Traefik	Modern, easy to use, great for microservices.	Auto-discovery, dashboard, Let's Encrypt.
HAProxy Ingress	Based on HAProxy Load Balancer.	High availability, TCP/HTTP routing.
AWS ALB Ingress Controller	Uses AWS Application Load Balancer.	AWS native integration.

> Install an Ingress Controller before creating Ingress resources.

5 Ingress Features

Host-based Routing	Route traffic based on domain names.
Path-based Routing	Route traffic based on URL paths.
TLS / SSL Termination	Terminate HTTPS at the Ingress Controller.
Annotations	Customize controller behavior.
URL Rewriting	Rewrite URLs before sending to backend.
Rate Limiting	Control traffic rate (with annotations).

6 Example Flow



7 Useful Commands

```
kubectl get ingress                          # List all ingresses
kubectl describe ingress my-ingress          # Describe ingress
kubectl get ingress -n default               # List in namespace
kubectl get ingress -o wide                  # Show IP/Host
kubectl get ingressclass                     # List ingress classes
kubectl logs -n ingress-nginx \              # Controller logs
  deployment/ingress-nginx-controller
```

8 Best Practices

- ☑ Use Ingress instead of exposing Services with NodePort.
- ☑ Always use TLS/HTTPS for production.
- ☑ Define specific hostnames instead of relying on IPs.
- ☑ Use pathType: Prefix unless you need exact match.
- ☑ Keep Ingress rules simple and organized.
- ☑ Monitor Ingress Controller logs and metrics.

9 Key Takeaways

- ☑ Ingress exposes HTTP/HTTPS routes to Services.
- ☑ It uses Ingress Controller to route traffic.
- ☑ Supports host-based and path-based routing.
- ☑ Requires an Ingress Controller to work.
- ☑ Adds features like SSL, URL rewrite, and rate limiting.
- ☑ Makes applications accessible and secure.

Ingress is the gateway to your Kubernetes apps!

Great apps need a great entry point. Use Ingress to route smarter! ☺

StatefulSets - Stateful Applications Made Easy

StatefulSets

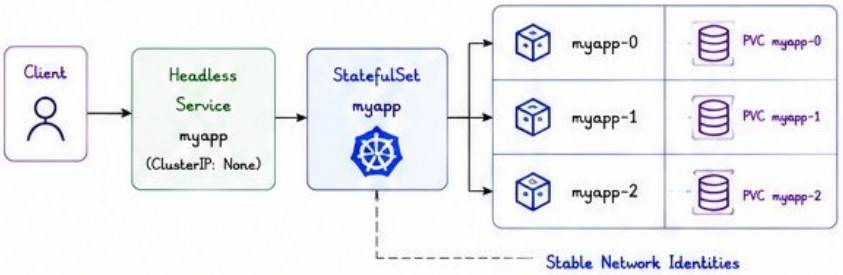
StatefulSets manage stateful applications that require stable network identities, ordered deployment, and persistent storage.

1 What is a StatefulSet?

- Manages stateful applications.
- Provides stable, unique network identities to Pods.
- Guarantees ordered, graceful deployment and scaling.
- Each Pod gets its own persistent storage that persists across restarts and rescheduling.
- Ideal for databases, message brokers, distributed systems.

Think:
If Deployments are for stateless apps, StatefulSets are for stateful apps.

2 StatefulSet Architecture



Uses a Headless Service (ClusterIP: None) to create stable DNS entries:
`<pod-name>.<service-name>.<namespace>.svc.cluster.local`

3 Pod Identity & DNS

Headless Service
myapp
(None)

Pod Name	DNS Record
myapp-0	myapp-0.myapp.default.svc.cluster.local
myapp-1	myapp-1.myapp.default.svc.cluster.local
myapp-2	myapp-2.myapp.default.svc.cluster.local

Stable network IDs are essential for databases and distributed systems.

4 Ordered Deployment & Scaling

Deployment Order	Deletion/Scaling Order
1. Create myapp-0	1. Delete myapp-2
2. Create myapp-1	2. Delete myapp-1
3. Create myapp-2	3. Delete myapp-0

Ensures data consistency and proper quorum in clusters.

5 Persistent Storage

- Each Pod gets its own PVC.
- PVCs are bound to specific Pods and not shared.
- Data survives Pod restarts, rescheduling, and upgrades.

6 StatefulSet YAML Example

YAML

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        ports:
        - containerPort: 3306
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: fast-ssd
      resources:
        requests:
          storage: 10Gi
```

Explanation

- `serviceName`: Headless Service used for stable DNS.
- `replicas`: Number of Pods.
- `volumeClaimTemplates`: Automatically creates a PVC for each Pod.
- Each Pod gets a unique PVC: data-mysql-0, data-mysql-1...

Best Practice

- ☑ Use a Headless Service.
- ☑ Use volumeClaimTemplates.
- ☑ Choose the right StorageClass.
- ☑ Monitor disk and I/O.
- ☑ Backup your data regularly.

7 StatefulSet vs Deployment

Feature	StatefulSet	Deployment
Purpose	Stateful applications	Stateless applications
Pod Identity	Stable, unique	Ephemeral
Storage	Persistent (per Pod)	Usually none
DNS	Stable DNS entries	Dynamic DNS
Deployment Order	Ordered (one by one)	Parallel
Scaling	Ordered	Parallel
Use Cases	Databases, Brokers, Distributed Systems	Web apps, APIs, Microservices

9 Useful Commands

```
kubectl get statefulsets                    # List StatefulSets
kubectl get pods -l app=mysql               # List Pods
kubectl describe statefulset mysql          # Describe StatefulSet
kubectl get pvc                             # List PVCs
kubectl scale statefulset mysql --replicas=5  # Scale StatefulSet
kubectl exec -it mysql-0 -- mysql -u root -p  # Connect to Pod
kubectl delete statefulset mysql            # Delete StatefulSet
```

8 Common Use Cases

Databases (MySQL, PostgreSQL)	Databases (MongoDB)	Caching (Redis)	Message Brokers (Kafka)	Distributed Systems (Zookeeper, etcd)

Interview Tip

StatefulSets = Stable Identity + Ordered Operations + Persistent Storage

10 Key Takeaways

- ☑ StatefulSets are for stateful applications.
- ☑ Provide stable network identities and storage.
- ☑ Use Headless Service for stable DNS.
- ☑ Deployment and scaling are ordered.
- ☑ Never delete PVCs unless you want to lose data.
- ☑ Back up your data. Always!

Stateful applications need stability, not speed. That's what StatefulSets deliver! ☺

DaemonSets – One Pod on Every Node

DaemonSets

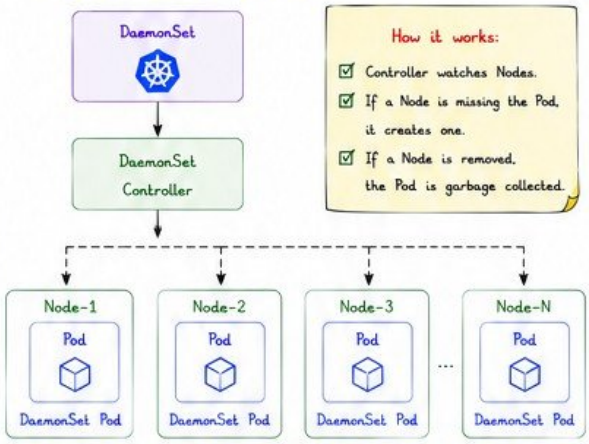
DaemonSets ensure that a copy of a Pod runs on all (or selected) Nodes in the cluster. They are ideal for cluster-level background tasks and node services.

1 What is a DaemonSet?

- Ensures one Pod runs on each Node (or subset of Nodes).
- Pods are added when new Nodes join and removed when Nodes leave.
- Commonly used for monitoring, logging, security, and networking agents.
- Managed by the DaemonSet controller.

Think:
DaemonSet = Background agent on every Node of your cluster.

2 DaemonSet Architecture



How it works:
- ☑ Controller watches Nodes.
- ☑ If a Node is missing the Pod, it creates one.
- ☑ If a Node is removed, the Pod is garbage collected.

3 Key Characteristics

One Pod per Node (or matched Nodes).	
Automatically handles Node joins & leaves.	
Supports Node Selectors & Taints.	
Ideal for cluster maintenance tasks.	
Does not create Pods on unschedulable Nodes.	

By default, DaemonSets run on all schedulable Nodes.

4 Use Cases

Monitoring Agents	(e.g., Prometheus Node Exporter)
Logging Agents	(e.g., Fluentd, Filebeat)
Security Agents	(e.g., Falco, Wazuh)
Networking Plugins	(e.g., Calico, Cilium)
Node Maintenance	(e.g., log rotation, cleanup)
System Level Daemons	(e.g., kube-proxy)

5 DaemonSet YAML Example

YAML

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    app: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
      - name: node-exporter
        image: prom/node-exporter:v1.7.0
        ports:
          - containerPort: 9100
            name: metrics
      tolerations:
        - operator: Exists
      hostNetwork: true
      hostPID: true
```

Explanation
- `selector`: Matches Pods managed by this DaemonSet.
- `hostNetwork: true` → Pod shares Node's network.
- `hostPID: true` → Access host process ID namespace.
- `tolerations`: Allows Pod to run on master/control plane Nodes (if tainted).

Tip
Use NodeSelector or Tolerations to control where DaemonSet Pods should run.

6 Scheduling & Node Selection

Run on specific Nodes using:
- ☑ NodeSelector
- ☑ Node Affinity
- ☑ Taints & Tolerations

Example: NodeSelector

```yaml
spec:
  template:
    spec:
      nodeSelector:
        disktype: ssd
```

Example: Tolerations

```yaml
tolerations:
- key: "node-role.kubernetes.io/control-plane"
- operator: "Exists"
  effect: "NoSchedule"
```

7 Lifecycle

1. DaemonSet created
2. Controller checks Nodes
3. Pod created on each matching Node
4. New Node joins → Pod auto created
5. Node removed → Pod removed

8 Useful Commands

```
kubectl get daemonsets -A                                  # List all DaemonSets
kubectl get pods -n monitoring -o wide                     # DaemonSet Pods
kubectl describe daemonset node-exporter -n monitoring     # Describe DaemonSet
kubectl edit daemonset node-exporter -n monitoring         # Edit DaemonSet
kubectl rollout restart daemonset node-exporter -n monitoring # Restart Pods
kubectl delete daemonset node-exporter -n monitoring       # Delete DaemonSet
kubectl logs -n monitoring -l app=node-exporter --tail=50  # View Logs
```

9 Best Practices

- ☑ Use DaemonSets for node-level tasks only.
- ☑ Use NodeSelector / Affinity to target Nodes.
- ☑ Add Tolerations if needed for control-plane Nodes.
- ☑ Monitor DaemonSet Pods for failures.
- ☑ Keep resource usage minimal.
- ☑ Ensure configuration is idempotent.

10 DaemonSet vs Deployment

Feature	DaemonSet	Deployment
Pods per Node	One Pod on each Node	Multiple Pods on selected Nodes
Use Case	Node-level tasks	Application workloads
Scaling	Matches number of Nodes	Manually / Autoscaled
New Node Join	Pod created automatically	No automatic action
Update Strategy	Rolling update per Node	Rolling update across Pods
Removal	Pod removed when Node removed	Pod removed based on replicas

Interview Questions
- What is a DaemonSet?
- How is it different from Deployment?
- When would you use DaemonSet instead of Deployment?
- Does DaemonSet run on all Nodes by default?
- How do you run DaemonSet only on worker Nodes?
- What happens when a new Node joins the cluster?

Key Takeaways
- ☑ DaemonSets run one Pod on every (or selected) Node.
- ☑ Ideal for monitoring, logging, security, and networking.
- ☑ Automatically handles Node lifecycle events.
- ☑ Use selectors and tolerations to control placement.
- ☑ Lightweight, powerful, and essential for cluster ops!

DaemonSets keep your cluster healthy, visible, and secure – one Node at a time! 🚀

Jobs & CronJobs – Run Tasks, Not Forever!

Jobs & CronJobs

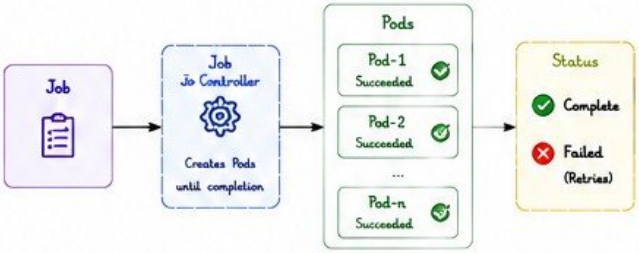
Jobs create Pods that run to completion. CronJobs schedule Jobs to run at recurring times. Perfect for batch processing, backups, and automation.

1 Jobs – Run to Completion

- Creates one or more Pods and ensures they successfully complete.
- Retries failed Pods based on backoff policy.
- Ideal for batch processing, data migration, and tasks that exit.
- Once succeeded, Pods are marked as Completed.

Think:
Jobs = Finite tasks that finish.
Deployments = Infinite tasks that keep running.

2 Jobs Architecture



The Job Controller ensures the required completions are met. If a Pod fails, a new Pod is created.

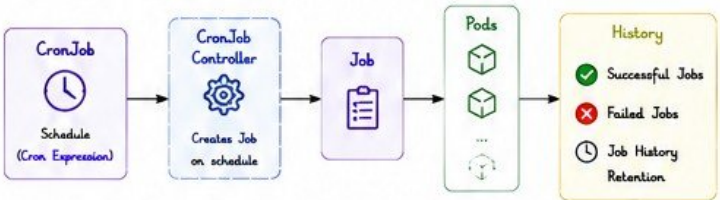
3 Common Use Cases

Batch processing
Data migration
Database backups
Report generation
One-time tasks
ML model training

4 CronJobs – Schedule Jobs

- CronJobs create Jobs on a schedule.
- Uses cron expressions (same as Linux crontab).
- Great for recurring tasks like backups or cleanup.
- Keeps history of successful and failed Jobs.

5 CronJobs Architecture



CronJob → Job → Pods (Run & Complete).

6 Job YAML Example

YAML

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: backup-job
spec:
  completions: 1      # Successful completions required
  parallelism: 1      # Pods running in parallel
  backoffLimit: 3     # Retry limit on failure
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: backup
          image: busybox
          command: ["/bin/sh", "-c"]
          args:
          - echo "Backing up data..."; \
            sleep 10; \
            echo "Backup completed"
```

Job Fields

- completions: How many Pods must succeed.
- parallelism: How many Pods run at the same time.
- backoffLimit: Retries on failure.
- restartPolicy: Always set to Never.

7 CronJob YAML Example

YAML

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-report
spec:
  schedule: "0 2 * * *"          # At 02:00 AM every day
  timeZone: "UTC"
  startingDeadlineSeconds: 300   # Missed schedule window
  concurrencyPolicy: Forbid      # Allow | Forbid | Replace
  successfulJobsHistoryLimit: 3  # Keep last 3 successful jobs
  failedJobsHistoryLimit: 1      # Keep last 1 failed job
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: report
              image: alpine:3.19
              command: ["/bin/sh", "-c"]
              args:
              - echo "Generating daily report..."; \
                sleep 10; \
                echo "Report generated"
```

Cron Syntax

```
* * * * *
| | | | └── Day of week
| | | └──── Month
| | └────── Day of month
| └──────── Hour (0-23)
└────────── Min. (0-59)
```

Example:
0 2 * * *
At 02:00 AM daily

Tips

- ☑ Use UTC timezone by default.
- ☑ Set history limits to avoid clutter.

8 Job Statuses

Status	Description
Active	Job is running.
Succeeded	Job completed successfully.
Failed	Job failed after retries exceeded.
Complete	All required completions met.

9 Concurrency Policy (CronJob)

Policy	Meaning
Allow	Start a new Job even if previous is running.
Forbid	Skip new Job if previous hasn't finished.
Replace	Terminate running Job and start a new one.

10 Useful Commands

```
kubectl get jobs                          # List Jobs
kubectl describe job <job-name>           # Describe Job
kubectl logs job/<job-name>               # View Job logs
kubectl get cronjobs                      # List CronJobs
kubectl describe cronjob <name>           # Describe CronJob
kubectl get jobs --from=cronjob/<name>    # Jobs created by CronJob
kubectl delete job <job-name>             # Delete Job
kubectl delete cronjob <name>             # Delete CronJob
```

11 Best Practices

- ☑ Use Jobs for finite workloads only.
- ☑ Set appropriate backoffLimit.
- ☑ Use CronJob history limits.
- ☑ Use clear and consistent naming.
- ☑ Monitor Job and CronJob status.
- ☑ Keep tasks idempotent where possible.

Interview Questions

- What is the difference between a Job and a Deployment?
- What happens if a Job Pod fails?
- How does a CronJob create Jobs?
- What is the purpose of backoffLimit?
- What are the concurrency policies in CronJob?
- How do you view logs of a Job?
- How do you clean up old completed Jobs?

Key Takeaways

- ☑ Jobs run tasks to completion.
- ☑ CronJobs schedule Jobs on a time.
- ☑ Use Jobs for batch & one-time tasks.
- ☑ CronJobs for recurring automation.
- ☑ Monitor, limit history, and clean up.

Quick Recall

Job = Run now, finish.
CronJob = Run later, repeat.
Completion = Success.
BackoffLimit = Retry attempts.
ConcurrencyPolicy = Control overlaps.

Pro Tip

Keep your Jobs small, deterministic, and idempotent for reliable automation!

Automate smartly. Schedule reliably. Let Kubernetes handle the rest!

Kubernetes Networking – Connect Everything

Networking

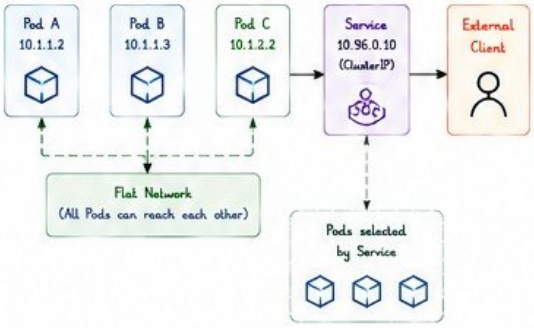
Kubernetes Networking enables communication between Pods, Services, and external clients across the cluster and the world.

1 Networking Goals

- All Pods can communicate with all other Pods.
- Pods can communicate with Services.
- External clients can access Services.
- Network policies control traffic.
- Highly available and scalable networking.

Think:
Kubernetes gives Pods their own IP and a flat network so they can talk to each other easily.

2 Kubernetes Networking Model



3 Service Types & Access

Type	Scope	How to Access
ClusterIP (Default)	Internal within cluster	Cluster IP 10.96.x.x
NodePort	External via Node IP & Port	<NodeIP>:<NodePort> (30000-32767)
LoadBalancer	External via Cloud LB	External IP from Cloud Provider
ExternalName	Maps to external DNS	myservice.example.com

4 CNI – Container Network Interface

- CNI plugins provide networking for Pods.
- Popular CNI plugins: Calico, Cilium, Flannel, Weave Net.
- CNI handles IPAM, routing, and connectivity.

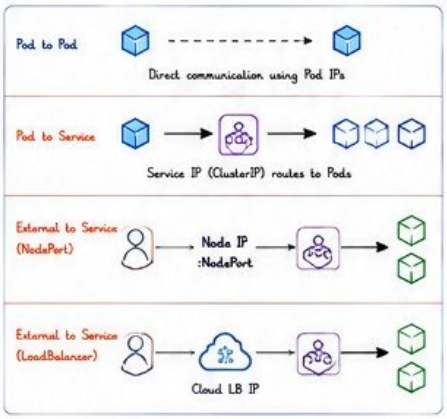
How CNI Works?

1. Assigns IP to Pods
2. Configures networking on Nodes
3. Enables Pod-to-Pod communication
4. Enforces Network Policies

Popular CNI Plugins

CALICO | cilium | flannel | weavenet

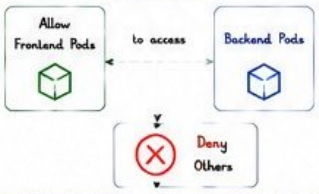
5 Networking Flow



6 Network Policies

- Control traffic at Pod level.
- Applied using labels and rules.
- Default: Allow all (unless policy exists).

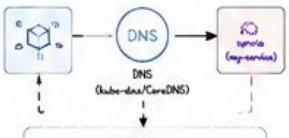
Example Rule



Benefits

- Improve security.
- Isolate workloads.
- Allow only required communication.

7 DNS in Kubernetes



- CoreDNS is the default DNS server in Kubernetes.
- Services get DNS names automatically.
- Format: <service>.<namespace>.svc.cluster.local

8 Ports in Kubernetes

Port Range	Purpose
1 - 1023	Well-known ports (system)
1024 - 29999	User ports
30000 - 32767	NodePort Services
32768 - 60999	Dynamic / Private use

9 Ingress vs Service

Feature	Service	Ingress
Layer	L4 (TCP/UDP)	L7 (HTTP/HTTPS)
Routing	Based on IP & Port	Based on Host & Path
SSL Termination	No (Limited)	Yes
Use Case	Cluster communication	External HTTP/HTTPS

10 Common Network Paths

Pod → Pod	Direct using Pod IP
Pod → Service	Via ClusterIP
External → NodePort	NodeIP:NodePort
External → LoadBalancer	LB IP (Cloud)
External → Ingress	Ingress Controller → Service → Pods

11 Useful Commands

```
kubectl get pods -o wide                          # Show Pod IPs and Nodes
kubectl get svc                                   # List Services
kubectl get endpoints                             # Show Service Endpoints (Pods)
kubectl run netshoot --image=nicolaka/netshoot -it --rm # Debug Pod
kubectl exec -it <pod> -- ping <pod-ip>           # Test connectivity
kubectl exec -it <pod> -- nslookup <service-name> # Test DNS resolution
kubectl get networkpolicy -A                      # List Network Policies
kubectl describe networkpolicy <name>             # Describe Policy
```

12 Best Practices

- Use a reliable CNI plugin.
- Apply Network Policies.
- Use Ingress for HTTP/HTTPS.
- Prefer ClusterIP for internal communication.
- Monitor network performance.

13 Interview Questions

- How do Pods communicate with each other?
- What is the purpose of a CNI plugin?
- What is the difference between ClusterIP and NodePort?
- How does DNS work in Kubernetes?
- What is a Network Policy and why is it used?
- How does Ingress route traffic?

14 Key Takeaways

- Kubernetes provides a flat network for all Pods.
- Services provide stable endpoints for Pods.
- CNI plugins power Pod networking.
- Network Policies secure communication.
- Ingress exposes HTTP/HTTPS services.

Pro Tip

A strong network design = Secure, reliable, and scalable Kubernetes applications!

Great applications start with great connectivity. Design it right!

Scaling in Kubernetes – Scale Smart, Stay Reliable!

Scaling

Scaling ensures your applications handle more load by adding or removing resources automatically.
Kubernetes supports Horizontal, Vertical, and Cluster Autoscaling.

1 Why Scale?

- Handle increased traffic and load.
- Improve application availability and performance.
- Optimize resource utilization and reduce costs.
- Scale up during peak, scale down during idle.

Think:
Right resource at the right time = Happy users + Efficient cluster!

2 Types of Scaling

Horizontal Pod Autoscaler (HPA)
Scales the number of Pods. (Scale Out / In)

Vertical Pod Autoscaler (VPA)
Adjusts CPU/Memory requests & limits of Pods.

Cluster Autoscaler (CA)
Adds or removes Nodes in the cluster based on demand.

3 Scaling Flow (HPA Example)

Users / Traffic
↓
Metrics (CPU/Memory) collected by Metrics Server
↓
HPA (Evaluates metrics)
↓
Scale Deployment (Add/Remove Pods)
↓
Application handles the load

4 Horizontal Pod Autoscaler (HPA)

How HPA Works

1. Metrics Server collects resource usage.
2. HPA checks metrics against target values.
3. HPA updates the number of Pods in the target Deployment/ReplicaSet.

Example

Current: 2 Pods → High CPU → Scaled: 5 Pods

Key Terms

- minReplicas: Minimum number of Pods.
- maxReplicas: Maximum number of Pods.
- targetCPUUtilizationPercentage: Target average CPU usage.
- targetMemoryUtilizationPercentage: Target average Memory usage.

5 Vertical Pod Autoscaler (VPA)

How VPA Works

- Monitors resource usage of Pods.
- Recommends or updates CPU/Memory requests and limits.
3. Evicts Pods (if needed) so new Pods start with right resources.

VPA Modes

Off	Only provides recommendations.
Initial	Applies recommendations at Pod creation.
Auto	Automatically updates resources and restarts Pods.

6 Cluster Autoscaler (CA)

How CA Works

- Monitors pending Pods.
- If Pods can't be scheduled due to insufficient nodes, CA adds new Nodes.
- If Nodes are underutilized, CA removes them.

Pending Pods → Cluster Autoscaler → Add Node → Schedule Pods

Requirements

- ☑ Cloud provider integration
- ☑ Proper Node labels and groups
- ☑ Permissions for Node management

7 Scaling Comparison

Feature	HPA	VPA	Cluster Autoscaler
What it scales	Number of Pods	CPU/Memory per Pod	Number of Nodes
Scope	Application level	Pod level	Cluster level
Direction	Scale Out / In	Scale Up / Down	Scale Out / In
Use Case	Increase capacity for traffic	Right size resource requests	Handle more Pods at cluster level
Requires Restart	No	Yes (if Auto)	No
Depends on	Metrics Server	Metrics Server	Cloud Provider

8 HPA YAML Example

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

Explanation

- Scales the Deployment "webapp".
- Min Pods: 2
- Max Pods: 10
- Keeps average CPU usage around 60%.
- Uses Metrics Server for metrics.

9 Useful Commands

```
kubectl get hpa                          # List HPAs
kubectl describe hpa <name>              # Describe HPA
kubectl get vpa                          # List VPAs
kubectl describe vpa <name>              # Describe VPA
kubectl get nodes                        # List Nodes
kubectl top pods                         # Pod resource usage
kubectl top nodes                        # Node resource usage
kubectl get --raw \                      # Raw metrics
  /apis/metrics.k8s.io/v1beta1/pods
kubectl autoscale deployment webapp \
  --cpu-percent=60 --min=2 --max=10      # Create HPA
```

10 Best Practices

- ☑ Set appropriate min and max replicas.
- ☑ Use resource requests and limits for accurate scaling.
- ☑ Ensure Metrics Server is installed.
- ☑ Monitor scaling events and metrics.
- ☑ Test scaling under load.
- ☑ Use Cluster Autoscaler for cost optimization.

11 Common Use Cases

Web Apps (Traffic spikes)	E-commerce (Seasonal peaks)	APIs & Microservices	Batch Jobs (Data processing)	Analytics Workloads

12 Interview Questions

- What is the difference between HPA and VPA?
- What does Cluster Autoscaler do?
- What is the role of Metrics Server?
- How does HPA decide to scale Pods?
- Can HPA scale a DaemonSet? Why or why not?

13 Key Takeaways

- ☑ HPA scales Pods based on load.
- ☑ VPA right-sizes Pod resources.
- ☑ Cluster Autoscaler scales Nodes.
- ☑ Scaling = Performance + Cost efficiency.
- ☑ Monitor, configure, and optimize!

Scale intelligently. Deliver reliably. Kubernetes makes it easy!

Kubernetes Security - Protect, Control, Secure

Security

Security in Kubernetes is all about Authentication, Authorization, and protecting your applications, data, and cluster.

1 Security Goals

- Authenticate users and services.
- Authorize what they can do.
- Secure cluster components.
- Isolate workloads.
- Protect data and secrets.
- Audit and monitor activities.

Think: Secure by default, least privilege access, and defense in depth.

2 Kubernetes Security Model

User / → Authentication (Who are you?) → Authorization (What can you do?) → API Server (Gatekeeper) → Cluster Resources

All requests go through the API Server, which authenticates and authorizes before allowing access.

3 Key Security Components

RBAC	Role-Based Access Control for fine-grained permissions.
Service Accounts	Identities for Pods to access the API.
Secrets	Store sensitive data like passwords, tokens.
Network Policies	Control traffic between Pods and namespaces.
Pod Security	Restrict privileged containers and actions.
Audit Logging	Record API requests for accountability.

4 RBAC - Role Based Access Control

Role (Permissions) get, list, watch create, update delete → RoleBinding (Associate) → User / Group / ServiceAccount

- **Role:** Defines permissions within a Namespace.
- **ClusterRole:** Defines permissions cluster-wide.
- **RoleBinding:** Grants Role to a user/group/SA in a Namespace.
- **ClusterRoleBinding:** Grants ClusterRole cluster-wide.

5 Service Accounts

Service Account (Identity) → Pod → API Server

Token Mount (Auto)

- Pods use ServiceAccount tokens to interact with API.
- Use custom ServiceAccounts and least privilage access.

6 Secrets - Protect Sensitive Data

Secrets store sensitive information in encrypted (base64 encoded) format.

Passwords · Tokens · TLS Certs · API Keys

Best Practices

- Never store secrets in images or code.
- Use Kubernetes Secrets or external tools like HashiCorp Vault.
- Enable Encryption at Rest.

7 Network Policies - Control Traffic

Pod A → Network Policy → Pod B

Network Policy → Pod C

Example Rules

- Allow traffic from frontend to backend.
- Deny all ingress except allowed.
- Allow DNS access only.

8 Pod Security Standards

Restrict what Pods can do.

Profile	Description	Use Case
Privileged	Unrestricted.	Not Recommended
Baseline	Restrict known privileged actions.	General workloads
Restricted	Highly restricted, secure by default.	Production workloads

Controls include:

- Privilage escalation
- Host access
- Capabilities
- Running as non-root
- Seccomp
- Read-only root filesystem

9 Security Best Practices

- Use least privilege (RBAC).
- Use ServiceAccounts, not default.
- Enable Network Policies.
- Keep cluster, nodes, and images updated.
- Scan images for vulnerabilities.
- Enable audit logging.
- Encrypt data at rest and in transit.
- Use Pod Security Standards or OPA/Gatekeeper.

10 RBAC YAML Example (Role + RoleBinding)

YAML

```yaml
# Role
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]

---
# RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: default
subjects:
- kind: User
  name: developer
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

11 NetworkPolicy YAML Example

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8888
```

12 Useful Commands

```
kubectl get roles,rolebindings -A          # List RBAC
kubectl auth can-i get pods                # Check your permission
kubectl get serviceaccounts -A             # List ServiceAccounts
kubectl get secrets                        # List Secrets
kubectl describe secret <name>             # View Secret details
kubectl get networkpolicies -A             # List NetworkPolicies
kubectl describe networkpolicy <name>      # View NetworkPolicy
kubectl auth reconcile -f rbac.yaml        # Apply/Update RBAC
kubectl get pod -o wide                    # View Pod details
kubectl get events                         # View events
kubectl logs -f audit.log                  # View audit logs
```

Enable Audit Logging

```
--audit-log-path=/var/log/k8s/audit.log
--audit-policy-file=/etc/kubernetes/audit-policy.yaml
```

13 Sommon Security Threats

Exposed API Server · Over-privileged Users/Pods · Untrusted Images · Secrets Leakage · Insecure Network

14 Interview Questions

- What is RBAC in Kubernetes?
- What is the purpose of ServiceAccounts?
- How do Network Policies work?
- How do you store secrets in Kubernetes?
- What are Pod Security Standards?
- How do you secure the Kubernetes cluster?

15 Key Takeaways

- Authenticate every user and service.
- Authorize with least privilege.
- Encrypt and protect sensitive data.
- Control network traffic.
- Monitor, audit, and keep updating.
- Security is a continuous process!

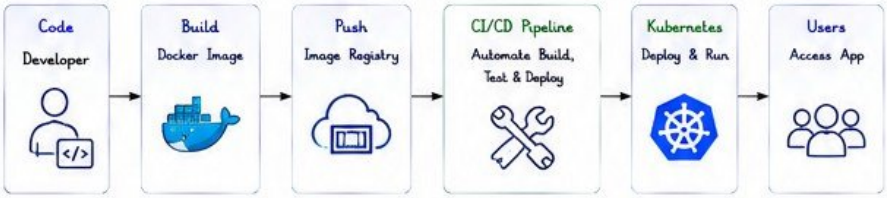
Secure your cluster today, sleep peacefully tomorrow.

Kubernetes + Docker + CI/CD – From Code to Cloud!

K8s + Docker + CI/CD

This page brings everything together! Build your app with Docker, automate with CI/CD, and deploy it to Kubernetes.

1 The Big Picture – From Code to Cloud



Code is built into a Docker image, pushed to a registry, and deployed to Kubernetes automatically using CI/CD.

2 Docker + Kubernetes

- Docker builds a portable image with everything your app needs.
- Kubernetes runs containers at scale, heals failures, and manages infrastructure.
- Together, they provide consistency, portability, and scalability.

Docker
- Build
- Ship
- Run anywhere

+

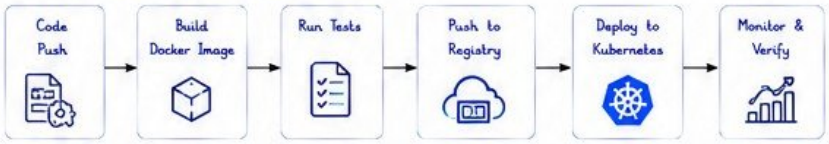
Kubernetes
- Orchestrate
- Scale
- Self-heal

3 Dockerfile Example

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python", "app.py"]
```

What it does?
- ☑ Starts from Python image
- ☑ Sets working directory
- ☑ Installs dependencies
- ☑ Copies app code
- ☑ Exposes port 8080
- ☑ Runs the application

4 CI/CD Pipeline Flow (Example)



Popular Tools

Source Control	CI Tools	Registry	CD / Deploy	Monitoring
GitHub | GitLab	Jenkins | GitHub Actions	Docker Hub | ECR | GCR	Argo CD | Helm	Prometheus | Grafana

5 Kubernetes Deployment YAML (Uses Docker Image)

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - name: webapp
        image: mydockerhub/webapp:1.0.0
        ports:
        - containerPort: 8080
        resources:
          limits:
            cpu: "500m"
            memory: "512Mi"
          requests:
            cpu: "250m"
            memory: "256Mi"
```

Explanation
- `replicas`: Run 3 Pods
- `image`: Uses Docker image from registry
- `resources`: Sets CPU & Memory limits/requests

Good Practice
Always use specific image tags (e.g., 1.0.0) not :latest in production!

6 Push Docker Image to Registry

Example using Docker Hub

```
# Build image
docker build -t mydockerhub/webapp:1.0.0 .

# Login
docker login

# Push image
docker push mydockerhub/webapp:1.0.0
```

Other Registries
- AWS ECR: `aws ecr get-login-password | docker login`
- GCP GCR: `gcloud auth configure-docker`
- Azure ACR: `az acr login --name <acr-name>`

7 Helm – Package Your App

YAML

Helm Chart Structure

```
mychart/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
└── charts/ (optional)
```

Why Helm?
- ☑ Package Kubernetes manifests
- ☑ Reuse and share charts
- ☑ Easy customization with values.yaml
- ☑ Versioning and rollbacks

Useful Commands

```
helm install myapp ./mychart
helm upgrade myapp ./mychart
helm list
helm uninstall myapp
```

8 Example End-to-End Flow

Automate this pipeline and ship faster with confidence! 🚀

9 Best Practices

- ☑ Use small, secure base images
- ☑ Scan images for vulnerabilities
- ☑ Use specific image tags
- ☑ Store secrets in Kubernetes Secrets
- ☑ Use resource limits and requests
- ☑ Automate everything – no manual steps!

10 Common Challenges

- ☒ Large images → slow builds & deploys
- ☒ Secrets in code → security risk
- ☒ No resource limits → unstable apps
- ☒ Manual deployments → human errors
- ☒ No monitoring → hard to troubleshoot

Automate. Containerize. Orchestrate. That's the DevOps way! 🚀

VERIQTA

Kubernetes Interview Questions + Cheat Sheet

Crack your K8s interview with confidence! Key questions, short answers, and must-know commands in one place.

Interview & Cheat Sheet

1 Interview Questions by Topic

Basics

1. What is Kubernetes?
2. What are the main components of Kubernetes?
3. What is a Pod?
4. What is the difference between Pod and Container?
5. What is a Namespace?

Workloads

1. What is a Deployment?
2. What is the difference between Deployment and ReplicaSet?
3. What is a StatefulSet? When would you use it?
4. What is a DaemonSet?
5. What are Jobs and CronJobs?

Networking

1. What is a Service in Kubernetes?
2. What are the types of Services?
3. What is Ingress?
4. How does Ingress work?
5. What is the difference between ClusterIP and NodePort?

Storage & Config

1. What is a PersistentVolume (PV)?
2. What is PersistentVolumeClaim (PVC)?
3. What is the difference between ConfigMap and Secret?
4. How do you mount a ConfigMap or Secret in a Pod?

Scaling & Performance

1. What is Horizontal Pod Autoscaler?
2. What is Vertical Pod Autoscaler?
3. What is Cluster Autoscaler?
4. How does HPA work?
5. When would you use VPA?

Security

1. What is RBAC in Kubernetes?
2. What are Roles and ClusterRoles?
3. What are Service Accounts?
4. How do you secure Secrets?
5. What are Network Policies?

Operations

1. How do you update an application in Kubernetes?
2. What is a Rolling Update?
3. How do you rollback a Deployment?
4. How do you troubleshoot a Pod that is not running?
5. How do you check resource usage in Kubernetes?

Advanced

1. What is the difference between StatefulSet and Deployment?
2. How does Kubernetes handle scheduling?
3. What are Taints and Tolerations?
4. What is the purpose of a Headless Service?
5. How does Kubernetes ensure high availability?

2 Kubernetes Cheat Sheet – Must Know Commands

Task	Command
View all Pods	`kubectl get pods`
View Pods (wide)	`kubectl get pods -o wide`
View all Services	`kubectl get svc`
View all Deployments	`kubectl get deployments`
View all Nodes	`kubectl get nodes`
View all Namespaces	`kubectl get ns`
Describe a resource	`kubectl describe <resource> <name>`
View Logs	`kubectl logs <pod-name>`
Follow Logs	`kubectl logs -f <pod-name>`
Execute into a Pod	`kubectl exec -it <pod-name> -- /bin/sh`
Copy file from Pod	`kubectl cp <pod>:<path> <local-path>`
Scale a Deployment	`kubectl scale deployment <name> --replicas=3`
Delete a resource	`kubectl delete <resource> <name>`
Apply YAML file	`kubectl apply -f file.yaml`
Get Events	`kubectl get events --sort-by=.metadata.creationTimestamp`
Get all (all namespaces)	`kubectl get all -A`

3 Resource Types Quick Reference

Type	Purpose	Example
Pod	Smallest deployable unit (1 or more containers)	webapp-pod
Deployment	Manages ReplicaSets & Pods (Stateless apps)	webapp-deploy
ReplicaSet	Ensures specified number of Pod replicas	webapp-rs
StatefulSet	Manages stateful applications (unique identity, ordered)	mysql-statefulset
DaemonSet	Runs a Pod on every (or selected) Node	log-agent-ds
Service	Stable endpoint to access Pods	webapp-svc
Ingress	HTTP/HTTPS routing to Services	webapp-ingress
ConfigMap	Store non-sensitive configuration	app-config
Secret	Store sensitive data (passwords, tokens)	db-secret
PersistentVolume	Cluster storage resource	pv-volume
PersistentVolumeClaim	Requests storage from PV	pvc-claim
Namespace	Virtual cluster for resource isolation	dev-namespace

4 K8s Mnemonics

PODS = Portable, Operable, Disposable, Share nothing

KISS = Keep It Simple (Use Deployments)

DRY = Don't Repeat Yourself (Use Helm/CI-CD)

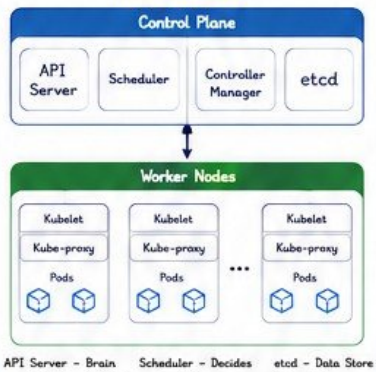
HPA = Handle Peak Automatically

SECURE = Service Accounts, Encrypt, Control, Use RBAC, Restrict, Enable audit

5 YAML Template (Quick Ref)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-container
        image: nginx:1.25
        ports:
        - containerPort: 80
```

6 Kubernetes Architecture (At a Glance)



7 Quick Tips

- ☑ Always use latest stable images.
- ☑ Set resource requests & limits.
- ☑ Use Namespaces for isolation.
- ☑ Enable liveness & readiness probes.
- ☑ Monitor with Prometheus & Grafana.
- ☑ Use Helm for package management.
- ☑ Automate with CI/CD pipelines.
- ☑ Keep your cluster & add-ons updated.

8 Key Takeaways

- ☆ Kubernetes Automates Deployment, Scaling & Operations.
- ☆ Understand the Building Blocks.
- ☆ Secure, Monitor & Optimize.
- ☆ Practice YAML & kubectl daily.
- ☆ **You've got this!** 💪

Learn. Practice. Deploy. Repeat. 🚀 You're now K8s Ready! 🎉